

Introdução a Algoritmos

Antonio Mauro Rezende, Ph.D.
Biotecnologia Aplicada ao Estudo de Patógenos
Instituto René Rachou, Fiocruz Minas



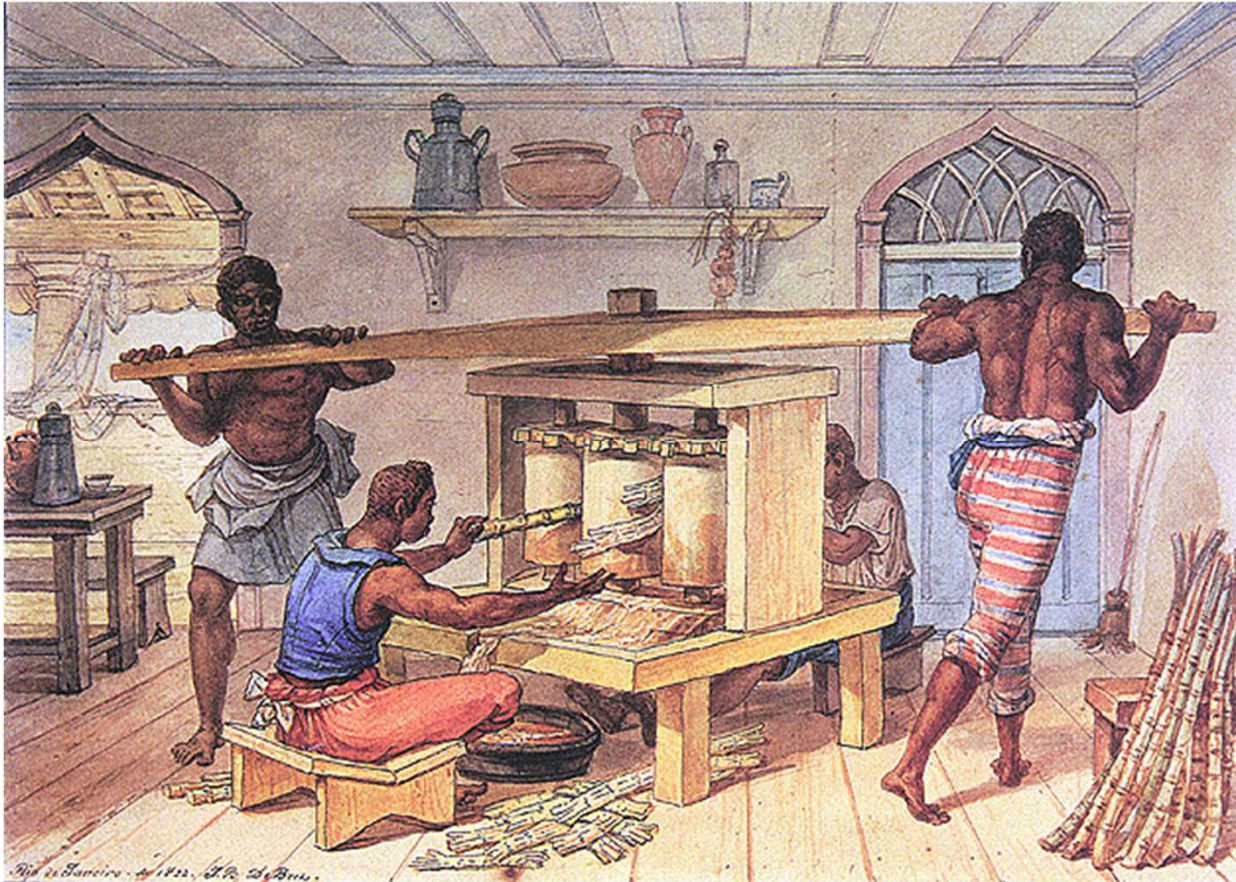
Disclaimers:

- O conteúdo dessa apresentação representa a minha opinião. Ele não representa o posicionamento das instituições citadas ao longo da minha fala.
- Apresentação foi criada com o suporte de ferramentas de IA, e eu me responsabilizo por todo conteúdo.

Lógica de programação

- Lógica – vem do grego *Logos* -> razão
- Relacionado ao raciocínio correto
- Para discernir entre idéias válidas, erros ou falácias

Lógica de programação



Qual o erro?

Fonte: CRUZ, Pedro Oswaldo. Disponível em: <http://enciclopedia.itaucultural.org.br/obra61279/engenho-manual-que-faz-caldo-de-cana>.

Lógica de programação

- Um raciocínio é expresso em palavras, e em qualquer um dos inúmeros idiomas existentes representará a mesma ideia
- Lógica de programação também: podemos utilizar diversas linguagens de programação para o mesmo raciocínio lógico

Lógica de programação



Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/1/5>

Lógica de programação



Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/1/5>

Lógica de programação

- Uma lista de comandos em uma determinada linguagem de programação seguindo uma lógica definida pode ser chamada de **ALGORITMO**
- Os computadores são extremamente “burros”
- A instrução deve estar clara e bem definida, caso contrário a máquina não lhe dará a resposta que você espera

Lógica de programação

A lógica no cotidiano do programador



<https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/1/5>

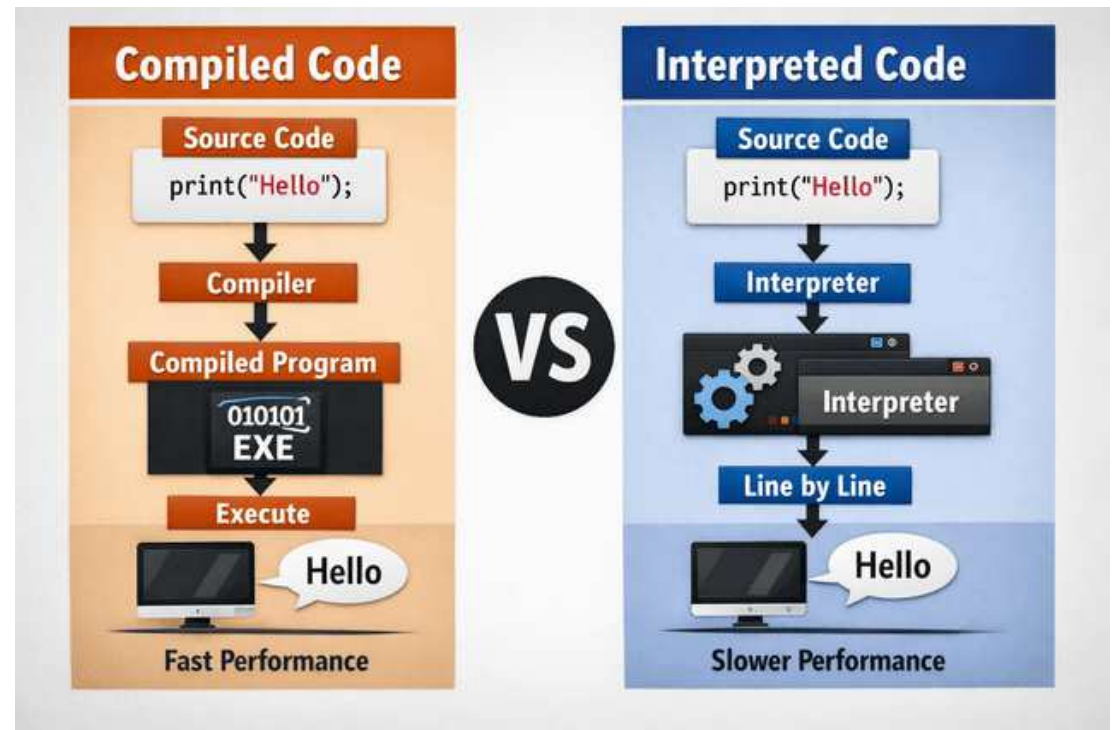
Lógica de programação



Essa é a linguagem que o computador entende:
Binária

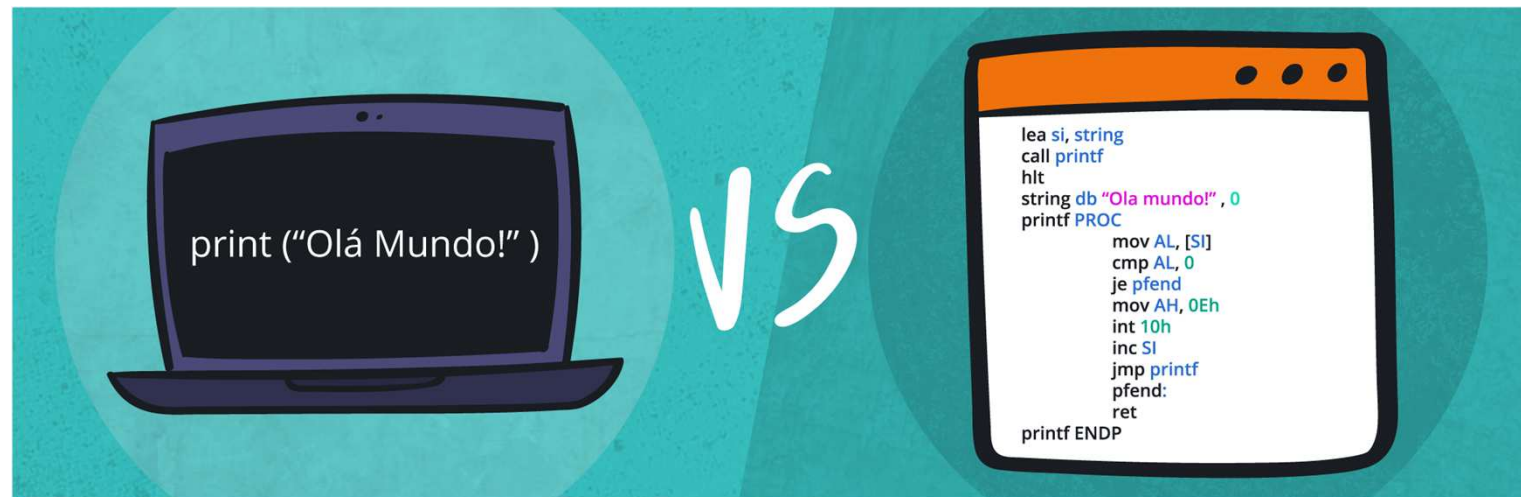
Lógica de programação

- Os comandos escritos em texto precisam ser convertidos para linguagem de máquina - binária
- Compilada vs Interpretada



Lógica de programação

- Os comandos escritos em texto precisam ser convertidos para linguagem de máquina - binária
- Alto nível vs Baixo nível
 - Java, Python
 - Assembler



Lógica de programação

- Cada linguagem de programação possui uma estrutura própria e suas palavras. A essa estrutura damos o nome de **sintaxe**.

Exemplo com linguagem Java

```
1 package br.ufrn.imd;  
2 public class Aula01 {  
3     public static void main(String[] args) {  
4         System.out.println("Exemplo de Código com Linguagem Java");  
5     }  
6 }
```

Exemplo com linguagem C

```
1 #include <stdio.h>  
2 int main(){  
3     printf("Exemplo de Código com Linguagem C\n");  
4     return 0;  
5 }
```

Exemplo com linguagem Potigol

```
1 escreva "Exemplo de Código com Linguagem Potigol"
```


Lógica de programação

- Os algoritmos possuem 3 elementos essenciais:
 - Entrada: dados
 - Processamento: sequência de passos executados para encontrar uma solução
 - Saída: Dados gerados pelo processamento
- Outros elementos importantes
 - Declaração de variáveis
 - Comandos de atribuição

Lógica de programação

- Variáveis
 - Regiões da memória RAM usadas para armazenar algum valor
 - Cada variável tem seu próprio endereço na memória
 - Tipagem Estática
 - Tipagem Dinâmica
 - Tipos básicos:
 - Inteiro (int): N=10
 - Ponto flutuante (float/double): N=10.456
 - Texto (string): N="aula"
 - Caractere (char): N="A"
 - Booleano (boolean): N=true

Lógica de programação

- Expressões e operadores
 - Uma associação de valores, variáveis, operadores ou funções que são avaliadas (processadas) de acordo com a sintaxe da linguagem e um valor é produzido
 - Operadores
 - Aritméticos
 - Relacionais
 - Lógicos

Lógica de programação

Quadro 01 - Operadores aritméticos e exemplos de expressões aritméticas

Lógica de programação

Operador	Operação	Exemplos de Expressões
+	Adição/Soma	$5 + 3$ $5 + a$
-	Subtração	$10 - 2$ $a - b$
*	Multiplicação	$c * d$ $2 * 36$
/ div	Divisão	$10 / 2$ x / y
mod	Resto da divisão	$10 \text{ mod } i$ $10 \text{ mod } 4$
^	Potência	2^3 e^f

Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/3/3>

Lógica de programação

Quadro 02 - Operadores relacionais e exemplos de expressões relacionais

Operador Relacional	Descrição	Exemplo
<	Menor que	5 < 3 a < b
<=	Menor ou igual	10 <= 2 a <= b
>	Maior que	c > d 2 > 36
>=	Maior ou igual	10 >= 2 x >= y
==	Igual	10 == i 10 == 4
<>	Diferente	2 <> 3 3 <> 3

Lógica de programação

Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/3/4>

Lógica de programação

Quadro 03 - Operadores lógicos

Nº Exp.	Valor de A	Valor de B	Operador ' e '	Operador ' ou '	' Não ' A	' Não ' B
1	Falso	Falso				
2	Verdadeiro	Falso				
3	Falso	Verdadeiro				
4	Verdadeiro	Verdadeiro				

Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/3/4>

Lógica de programação

- Precedência de Operadores

$$x = 2 + 3 * 4 - \frac{10}{2}$$

- X = 5 ou 9 ou 15

Quadro 04 - Precedências dos operadores aritméticos

Operador	Operação	Prioridade de Precedência
^	Exponenciação	0
*	Multiplicação	1
/	Divisão	1
mod	Resto da divisão	1
+	Adição	2
-	Lógica de programação	2

Posso mudar a ordem das operações?

Parênteses

$$X = 2^3 + 4/2$$

$$X = 2^{(3+4)}/2$$

Lógica de programação

- Estruturas de decisão



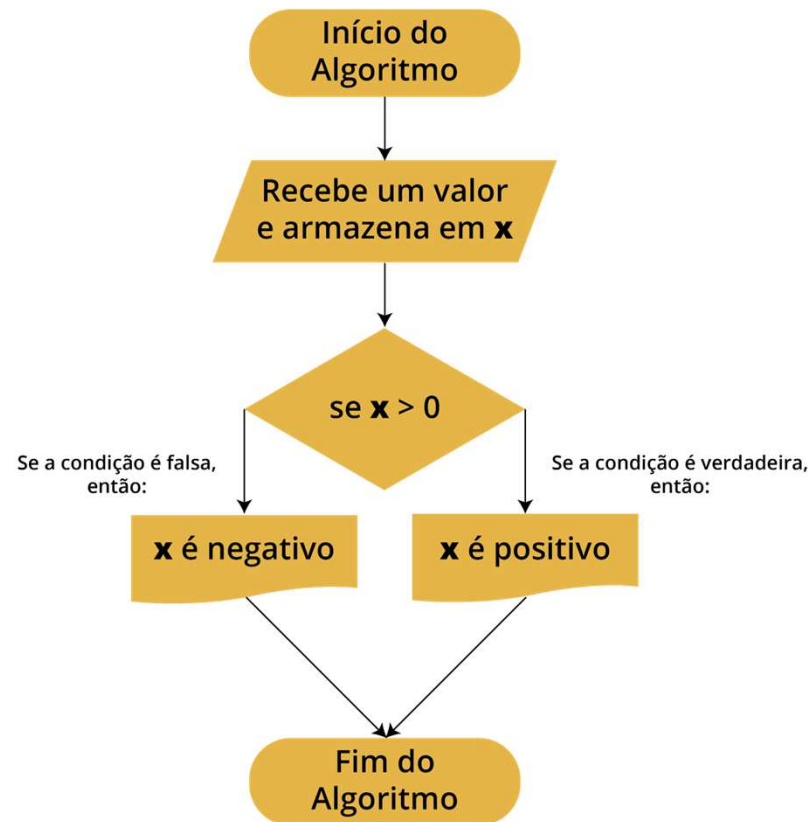
Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/3/7>

Lógica de programação

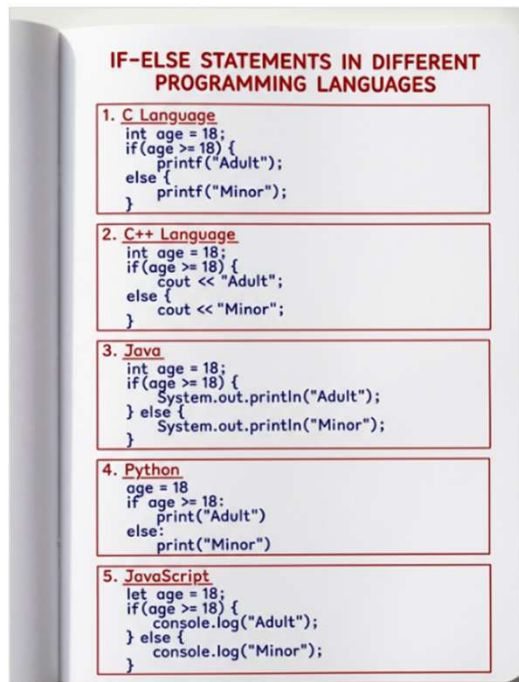
- Estruturas de decisão
 - Fundamentais em qualquer linguagem de programação e permitem criar algoritmos com diferentes de fluxo
 - Se uma condição é **verdadeira**, o algoritmo deverá seguir por um fluxo, se for **falsa** ele deverá, então, seguir por outro fluxo.

Lógica de programação

Expressões relacionais e as lógicas são utilizadas com frequência nas estruturas de decisão



Lógica de programação



1st Condition is true

```
let number = 2;  
if (number > 0) {  
    // code  
}  
else if (number == 0) {  
    // code  
}  
else {  
    //code  
}  
//code after if
```

2nd Condition is true

```
let number = 0;  
if (number > 0) {  
    // code  
}  
else if (number == 0) {  
    // code  
}  
else {  
    //code  
}  
//code after if
```

All Conditions are false

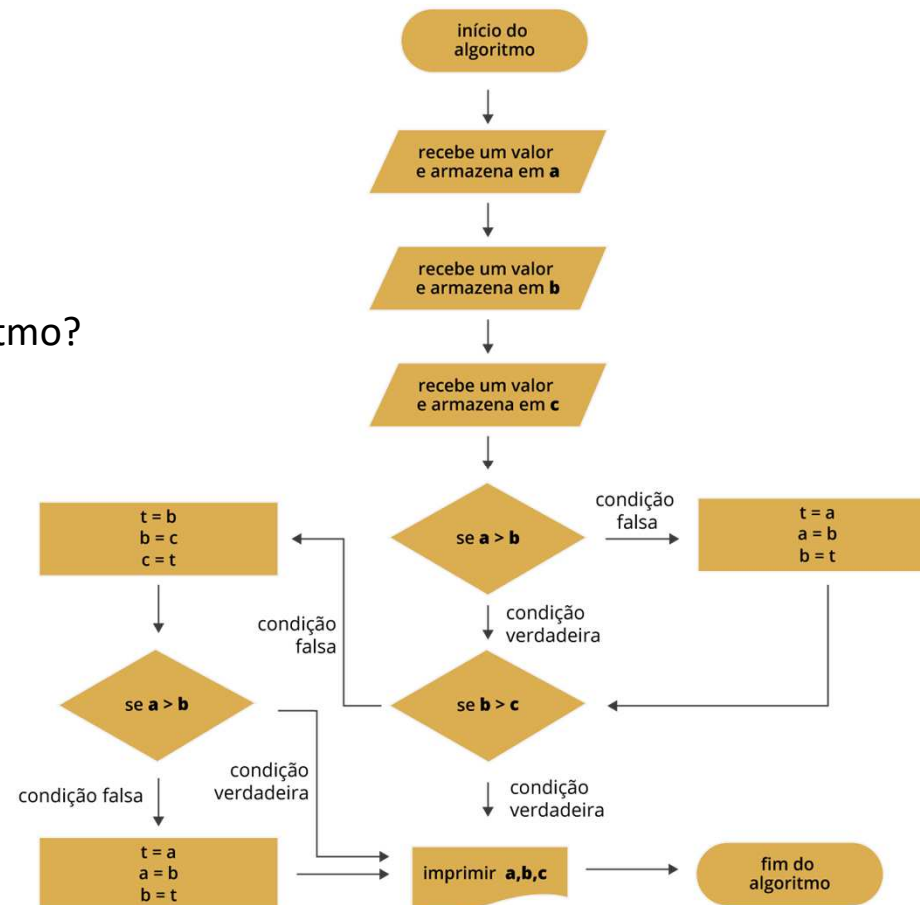
```
let number = -2;  
if (number > 0) {  
    // code  
}  
else if (number == 0) {  
    // code  
}  
else {  
    //code  
}  
//code after if
```

Fonte: <https://medium.com/geekculture/3-conditional-branching-alternatives-to-if-statements-db675e12936a>

Fonte: <https://www.instagram.com/p/DYNQr5yDnlv/>

Lógica de programação

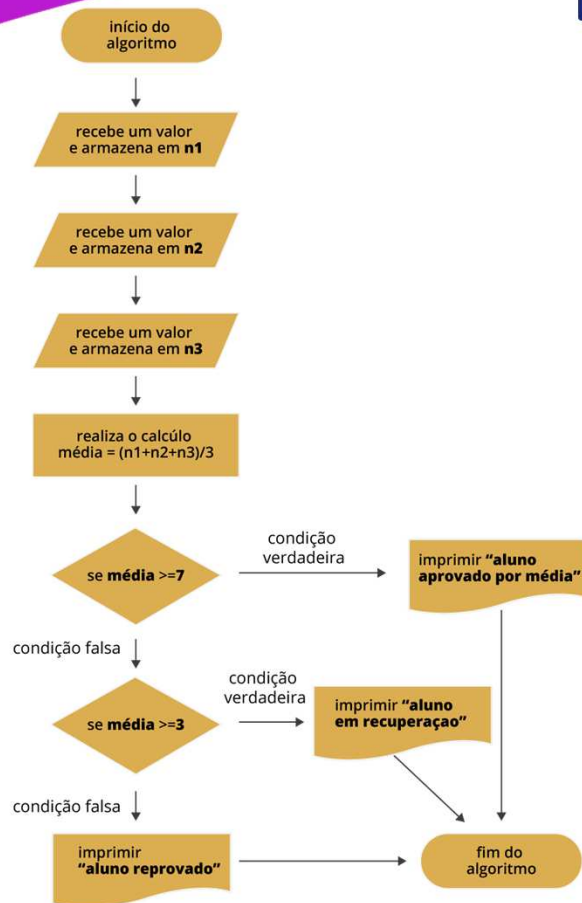
Para que serve esse algoritmo?



Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/4/3>

Lógica de programação

Aninhamento



```
1 escreva "Digite a primeira nota:"
2 var n1 := leia_numero
3 escreva "Digite a segunda nota:"
4 var n2 := leia_numero
5 escreva "Digite a terceira nota:"
6 var n3 := leia_numero
7
8 # lembre-se da precedência de operadores.
9 var media = (n1+n2+n3)/3
10
11 # primeira estrutura de decisão 'se'
12 se media >= 7 então
13     escreva "Aluno aprovado por média."
14 senão
15     # segunda estrutura de decisão 'se' dentro
16     # do bloco de condição falsa do primeiro 'se'
17     se media >= 3 então
18         escreva "Aluno em recuperação"
19     senão
20         escreva "Aluno reprovado"
21     fim
22 fim
23
24
```

Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/4/3>

Lógica de programação

```
function register()
{
    if (!empty($_POST)) {
        $msg = '';
        if ($_POST['user_name']) {
            if ($_POST['user_password_new']) {
                if ($_POST['user_password_new'] === $_POST['user_password_repeat']) {
                    if (strlen($_POST['user_password_new']) > 5) {
                        if (strlen($_POST['user_name']) < 65 && strlen($_POST['user_name']) > 1) {
                            if (preg_match('/^[a-z\d]{2,64}$/i', $_POST['user_name'])) {
                                $user = read_user($_POST['user_name']);
                                if (!isset($user['user_name'])) {
                                    if ($_POST['user_email']) {
                                        if (strlen($_POST['user_email']) < 65) {
                                            if (filter_var($_POST['user_email'], FILTER_VALIDATE_EMAIL)) {
                                                create_user();
                                                $_SESSION['msg'] = 'You are now registered so please login';
                                                header('Location: ' . $_SERVER['PHP_SELF']);
                                                exit();
                                            } else $msg = 'You must provide a valid email address';
                                        } else $msg = 'Email must be less than 64 characters';
                                    } else $msg = 'Email cannot be empty';
                                } else $msg = 'Username already exists';
                            } else $msg = 'Username must be only a-z, A-Z, 0-9';
                        } else $msg = 'Username must be between 2 and 64 characters';
                    } else $msg = 'Password must be at least 6 characters';
                } else $msg = 'Passwords do not match';
            } else $msg = 'Empty Password';
        } else $msg = 'Empty Username';
        $_SESSION['msg'] = $msg;
    }
    return register_form();
}
```

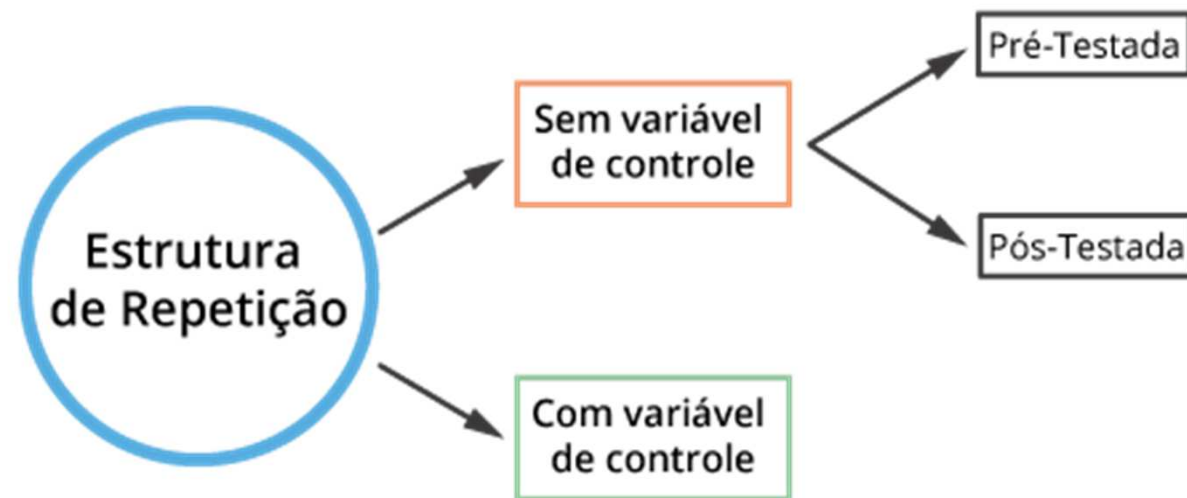


Fonte: <https://statelessmachine.com/p/the-big-short-circuit>

Lógica de programação

Estruturas de repetição

- Executa um conjunto de comandos por repetidas vezes até que uma condição não seja mais satisfeita



Lógica de programação

Estruturas de repetição

- Estrutura de repetição: **enquanto**
- Repetir uma mesma operação (ou conjunto de operações) enquanto uma condição lógica for **verdadeira**
- Estrutura sem variável de controle pré-testada

```
1 enquanto <condição> faça
2   <instruções>
3 fim
4
```

- Como seria um programa para escrever “Olá, seja bem-vindo!” 30 vezes

Lógica de programação

Estruturas de repetição

- Estrutura de repetição: **enquanto**
- Repetir uma mesma operação (ou conjunto de operações) enquanto uma condição lógica for **verdadeira**
- Estrutura sem variável de controle pré-testada
- Sem a linha 4, o que aconteceria?

```
1 var contador := 1
2 enquanto contador <= 30 faça # repete 'enquanto' o 'contador' for menor ou igual que 30
3   escreva "Olá, seja bem-vindo!"
4   contador := contador + 1 # a cada vez que escreve a mensagem, soma mais 1 no contador.
5 fim
6 escreva "Terminou!"
7
```

Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/6/4>

Lógica de programação

Estruturas de repetição

```
print "Informe um valor e pressione Enter\n";  
  
$soma = 0;  
$n=1;  
print "Digite 0 (zero) para encerrar o programa\n";  
  
while($n!=0){  
    $n=<STDIN>;  
    $soma=$soma+$n;  
}  
  
print "A soma total é $soma\n";
```

```
1 var soma := 0  
2 var n := 1  
3 escreva "Informe um valor e pressione Enter."  
4 escreva "Digite 0 (zero) para encerrar o programa."  
5 enquanto n <> 0 faça  
6     n := leia_inteiro  
7     soma := soma + n  
8 fim  
9  
10 escreva "A soma total é {soma}"
```

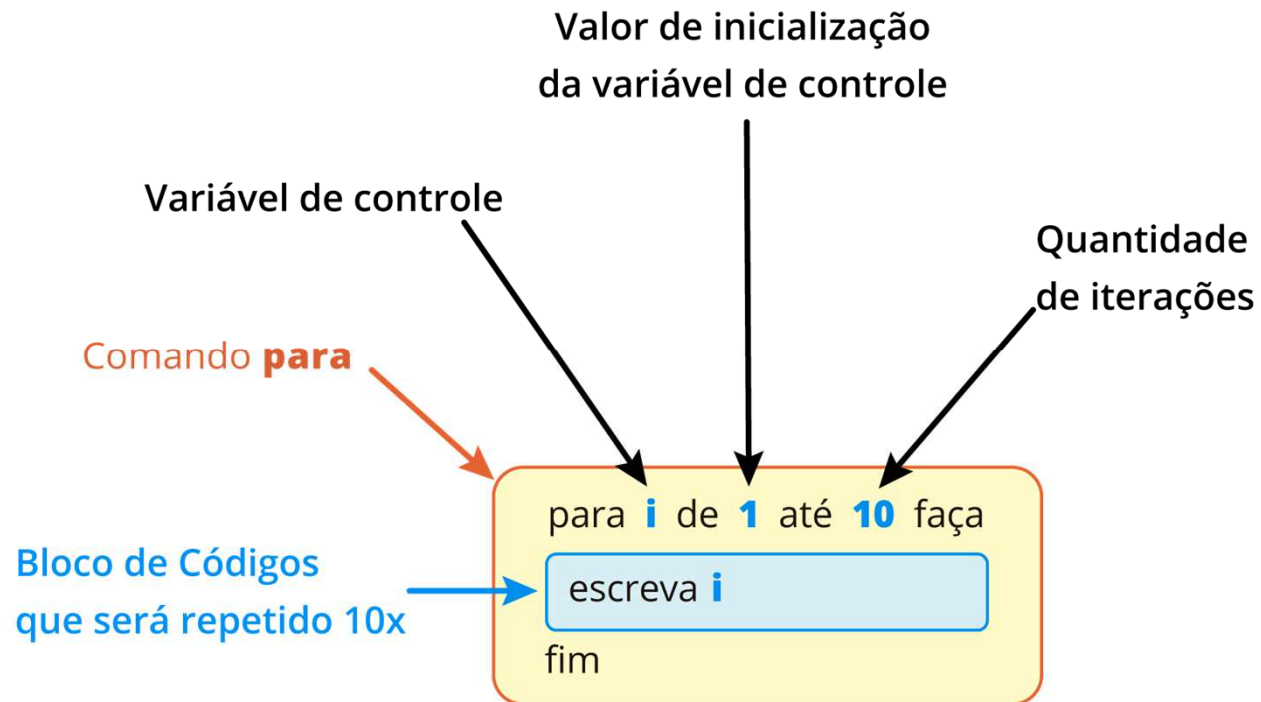
Lógica de programação

Estruturas de repetição

- Estrutura de repetição: **para**
- Repetir uma mesma operação com um número de vezes determinado
- Estrutura com variável de controle pré-testada

Lógica de programação

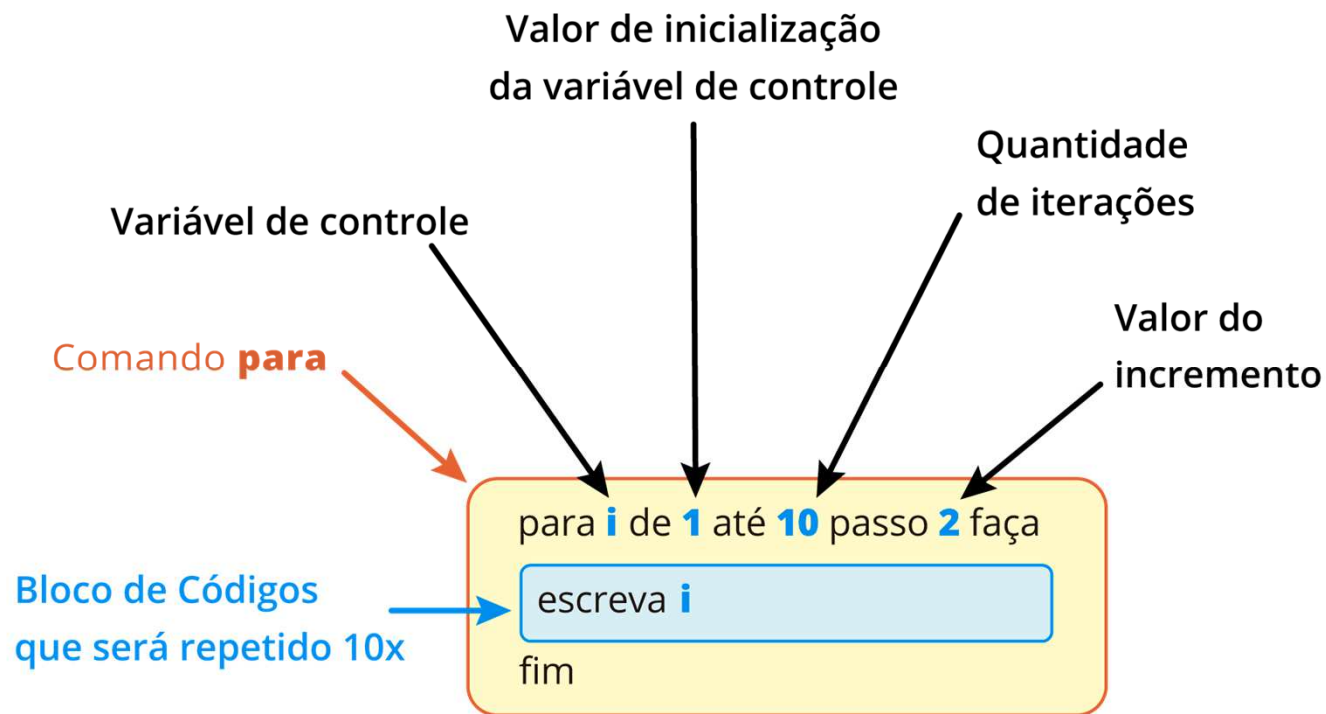
Estruturas de repetição



Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/7/4>

Lógica de programação

Estruturas de repetição



Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/7/5>

Lógica de programação

Estruturas de aninhadas

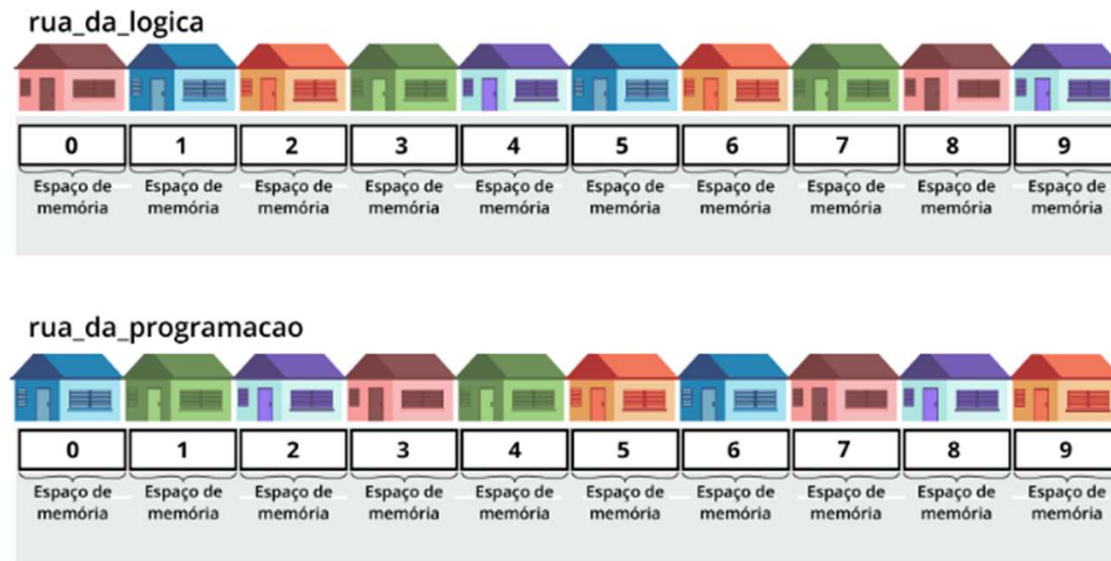
	30	40	50
<i>matriz</i> =	1	2	3
	1000	2000	3000

```
matriz = [[30,40,50],[1,2,3],[1000,2000,3000]]  
  
linhas = len(matriz)  
colunas = len(matriz[0])  
  
for i in range(1,linhas):  
    for j in range(1,colunas):  
        print(matriz[i][j])
```

Lógica de programação

Estruturas de dados

- Para construir programas precisamos de variáveis
- Se precisarmos de uma variável para armazenar diversos valores. Ex.: os 20 aminoácidos
- Nome e índice



Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/11/4>

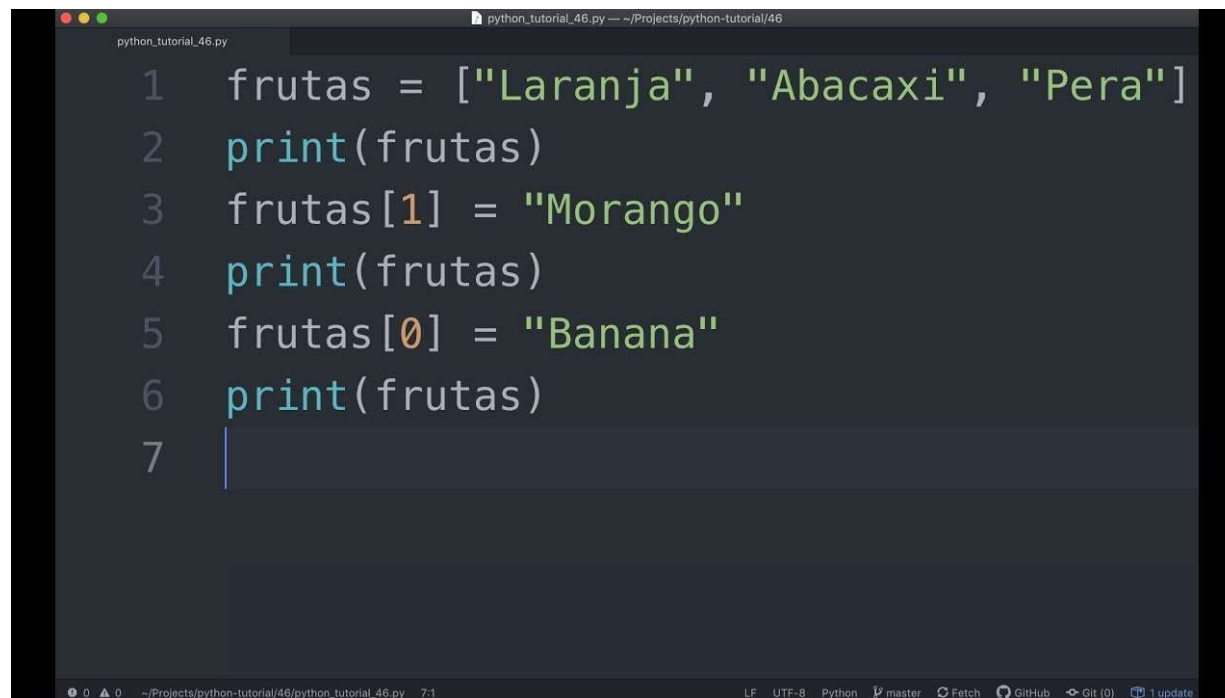
Lógica de programação

Estruturas de dados

- Exemplos de estrutura
 - Vetores (arrays)
 - Matrizes
 - Dataframe
 - Dicionários

Lógica de programação

Estruturas de dados - vetor



```
python_tutorial_46.py
1  frutas = ["Laranja", "Abacaxi", "Pera"]
2  print(frutas)
3  frutas[1] = "Morango"
4  print(frutas)
5  frutas[0] = "Banana"
6  print(frutas)
7  |
```

The screenshot shows a code editor window titled 'python_tutorial_46.py'. The code defines a list 'frutas' with three elements: 'Laranja', 'Abacaxi', and 'Pera'. It then prints the list, changes the second element to 'Morango', prints the list again, changes the first element to 'Banana', and prints the list a third time. The editor has a dark theme and a status bar at the bottom showing file encoding (UTF-8), language (Python), and branch (master).

Fonte: <https://www.youtube.com/watch?v=LYpVyBPehUw>

Lógica de programação

Estruturas de dados - matriz

	30	40	50
<i>matriz</i> =	1	2	3
	1000	2000	3000

```
matriz = [[30,40,50],[1,2,3],[1000,2000,3000]]  
  
linhas = len(matriz)  
colunas = len(matriz[0])  
  
for i in range(1,linhas):  
    for j in range(1,colunas):  
        print(matriz[i][j])
```

Lógica de programação

Estruturas de dados - dataframe

Dataframe

axis = 1

axis = 0

	nome	cidade	idade	nota
10	Pedro	São Paulo	34	83.0
11	Maria	São Paulo	23	59.0
12	Janaina	Rio de Janeiro	32	86.0
13	Wong	Brasília	43	89.0
14	Roberto	Salvador	38	98.0
15	Marco	Curitiba	31	61.0
16	Paula	Belo Horizonte	34	44.0

índice
(index)

axis = 0
'index'

axis = 1
'columns'

The diagram illustrates a DataFrame as a table. A vertical arrow on the left is labeled 'axis = 0' and points to the index column. A horizontal arrow at the top is labeled 'axis = 1' and points to the column headers. The index column is labeled 'índice (index)' and the column headers are labeled 'columns'.

Fonte: <https://forum.casadodesenvolvedor.com.br/topic/44842-maneiras-de-criar-um-dataframe/>

Lógica de programação

Estruturas de dados

```
1 escreva "Digite o 1º aluno:"
2 var aluno1 := leia_texto
3 escreva "Digite o 2º aluno:"
4 var aluno2 := leia_texto
5 escreva "Digite o 3º aluno:"
6 var aluno3 := leia_texto
7 escreva "Digite o 4º aluno:"
8 var aluno4 := leia_texto
9 escreva "Digite o 5º aluno:"
10 var aluno5 := leia_texto
11
12 escreva "Aluno: {aluno1}"
13 escreva "Aluno: {aluno2}"
14 escreva "Aluno: {aluno3}"
15 escreva "Aluno: {aluno4}"
16 escreva "Aluno: {aluno5}"
```

```
1 var alunos := vetor[10]
2
3 alunos[0] := leia_texto
4 alunos[1] := leia_texto
5 alunos[2] := leia_texto
6 alunos[3] := leia_texto
7 alunos[4] := leia_texto
8 alunos[5] := leia_texto
9 alunos[6] := leia_texto
10 alunos[7] := leia_texto
11 alunos[8] := leia_texto
12 alunos[9] := leia_texto
13
14 escreva "{alunos[0]}"
15 escreva "{alunos[1]}"
16 escreva "{alunos[2]}"
17 escreva "{alunos[3]}"
18 escreva "{alunos[4]}"
19 escreva "{alunos[5]}"
20 escreva "{alunos[6]}"
21 escreva "{alunos[7]}"
22 escreva "{alunos[8]}"
23 escreva "{alunos[9]}"
```

Lógica d

```
1 var alunos := vetor[10]
2
3 # ler 10 nomes.
4 para i de 0 até 9 faça
5     escreva "O nome do {i+1}º aluno:"
6     alunos[i] := leia_texto
7 fim
8
9 # imprimir os 10 nomes lidos
10 para i de 0 até 9 faça
11     escreva "{i+1}º aluno: {alunos[i]}"
12 fim
```



Lógica de programação

Funções, Parâmetros e Bibliotecas

- Quando um programa cresce, sua complexidade também aumenta
- O mesmo conjunto de instruções pode ser utilizados várias vezes no mesmo programa
- Organização em Funções
- É um subalgoritmo com nome, estrutura com início e fim definidos

Lógica de programação

Funções, Parâmetros e Bibliotecas

- Função
 - Nome: identificador pelo qual é chamada
 - Parâmetros: informações e dados que serão processados dentro da função
 - Variáveis: variáveis específicas da função e declaradas internamente
 - Escopo de variáveis: **global** e **local**

Lógica de programação

Funções, Parâmetros e Bibliotecas

```
1 soma(x: inteiro, y: inteiro) # Declaração de função com corpo
2   var d := x + y             # Comandos executados na função
3   retorne d                  # A última linha da função possui o comando de retorno
4 fim                           # O comando fim, ao final, define o 'limite' da função declarada
5
6 var a := 5
7 var b := 9
8 var c := 0
9
10 c := soma(a,b)
11
12 escreva "A soma de {a} + {b} é {c}"
```

Fonte: <https://materialpublic.imd.ufrn.br/curso/disciplina/4/70/14/4>

Lógica de programação

Convenção para pseudocódigo

- Estrutura geral (**INÍCIO/FIM**)
- Declaração de variáveis e tipos (**DECLARE, INTEIRO, REAL, CARACTERE, CADEIA DE CARACTERES**)
- Atribuição (**<-**)
- Entrada e saída (**LEIA, ESCREVA**)
- Operadores aritméticos (**+ - * / MOD**)
- Operadores relacionais (**== <> > < >= <=**)
- Operadores lógicos (**E, OU, NÃO**)
- Estruturas condicionais (**SE...ENTÃO...SENÃO...FIM SE**)
- Estruturas de repetição (**PARA...FAÇA / ENQUANTO...FAÇA**)
- Funções auxiliares (**COMPRIMENTO, acesso por índice**)

Lógica de programação

Convenção para pseudocódigo

Ex.:Determinar maior e menor número de uma sequência

INÍCIO

DECLARE quantidade, i, maior, menor : INTEIRO
DECLARE numeros : VETOR[1..100] DE INTEIRO

ESCREVA "Quantos números você deseja informar?"
LEIA quantidade

// Preenchimento do vetor
PARA i <- 1 ATÉ quantidade FAÇA
 ESCREVA "Digite o número ", i, ":"
 LEIA numeros[i]
FIM PARA

// Inicializa maior e menor com o primeiro elemento do vetor
maior <- numeros[1]
menor <- numeros[1]

// Percorre o vetor a partir do segundo elemento
PARA i <- 2 ATÉ quantidade FAÇA
 SE numeros[i] > maior ENTÃO
 maior <- numeros[i]

FIM SE

SE numeros[i] < menor ENTÃO
 menor <- numeros[i]

FIM SE
FIM PARA

ESCREVA "O maior número da sequência é: ", maior
ESCREVA "O menor número da sequência é: ", menor
FIM

Obrigado!

antonio.rezende@fiocruz.br

